

The can_gossip algorithm does **not** always find the shortest path.

It *will* find **a** path if one exists, but because of the way it explores the graph, that path can be longer than necessary.

From the slides you attached, you can already see a counter-example.

- We want a path from host **1** to host **5**.
- Look at the connections:
 - 1 is connected to 2 and 3
 - 2 is connected to 1, 3, and 4
 - 3 is connected to 1, 2, and 4
 - 4 is connected to 2, 3, and 5
 - 5 is connected to 4

If we only care about the *number of hops*, the **shortest** path from 1 to 5 is:

1 → 2 → 4 → 5 (3 hops)

However, the algorithm in can_gossip behaves like a **depth-first search**:

1. Start at 1. It arbitrarily chooses neighbor **2** first. Path: [1, 2]
2. At 2, it has neighbors {1, 3, 4}. It ignores 1 (already visited) and arbitrarily chooses **3** next. Path: [1, 2, 3]
3. At 3, neighbors are {1, 2, 4}. It ignores 1 and 2 (already visited) and goes to **4**. Path: [1, 2, 3, 4]
4. At 4, neighbors are {2, 3, 5}. It ignores 2 and 3 (already visited) and finally goes to **5**. Path: [1, 2, 3, 4, 5]

The algorithm stops as soon as it reaches 5, so the final path it finds is:

1 → 2 → 3 → 4 → 5 (4 hops)

This is **longer** than the optimal path (1 → 2 → 4 → 5).

The only reason it missed the shorter path is that, at node 2, it chose to go to 3 instead of 4. Because the algorithm gives “no preference” among neighbors and always dives as deep as possible before backing up, its result depends on that arbitrary neighbor order. When the neighbor order is unlucky, you still get a valid path, just not the shortest one.

To guarantee the shortest path in an unweighted network, we'd need a different strategy, such as **breadth-first search (BFS)**, which explores all nodes at distance 1 before distance 2, then distance 3, etc. The current can_gossip algorithm doesn't do that, so it cannot guarantee the shortest path between two hosts in every situation.